

Основы работы с объектом Event в JavaScript

В данном уроке мы изучим основы работы с объектом Event. Данный объект содержит в себе информацию о произошедшем событии. К примеру, если по элементу был сделан клик, мы можем узнать координаты этого клика, была ли нажата клавиша Ctrl, Alt или Shift в момент клика и так далее.

Давайте посмотрим, как получить объект Event. Пусть у нас есть кнопка:

```
1 <button id="elem">text</button>
```

Пусть по клику на эту кнопку выполняется некоторая функция:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function() {
4
5 });
```

Внутри привязанной функции уже есть объект Event - мы просто пока еще не знаем, как его получить. Получается он следующим образом: нужно при объявлении нашей функции передать в нее параметром любую переменную (как правило, event - но название может быть любым) и в эту переменную *браузер автоматически положит объект Event*:

```
1 elem.addEventListener('click', function(event) {
2   console.log(event); // увидим объект с событием
3 });
```

№ 1

⊗jsPmOEInr

Самостоятельно проделайте описанные манипуляции и выведите объект с событием в консоль. Изучите структуру этого объекта.

Координаты события в JavaScript

Изучите теорию по следующим ссылкам:

[clientX](#) [clientY](#) [pageX](#) [pageY](#)

№ 1

⊗jsPmOEEC

Сделайте так, чтобы при движении мышки по странице, отображались координаты курсора, подобно тому, как это сделано в следующем образце:

открыть в дочернем окне

Тип события в объекте Event в JavaScript

Объект Event также содержит в себе тип произошедшего события. Под типом понимается название click, mouseover и так далее. Тип события содержит свойство type:

```
1 elem.addEventListener('click', function(event) {
2   console.log(event.type); // выведет 'click'
3 });
```

№1

⊗jsPmOEET

В следующем коде к двум событиям привязан один и тот же обработчик:

```
1 <button id="elem">text</button>

1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', func);
4 elem.addEventListener('dblclick', func);
5
6 function func() {
7
8 }
```

Допишите код функции func так, чтобы при клике на элемент этот элемент красился в зеленый цвет, а при двойном клике - в красный.

Элемент события в объекте Event в JavaScript

Объект Event также позволяет получить элемент, в котором произошло событие. Этот элемент содержится в свойстве target. Зачем это нужно, если этот элемент содержится в this?

Дело в том, что на самом деле this всегда содержит элемент, к которому было привязано событие, а свойство target - элемент, по которому реально был клик. Этот реальный элемент может совпадать с this, а может не совпадать.

Давайте посмотрим на примере. Пусть у нас есть div, а внутри него абзац:

```
1 <div id="elem">
2   <p>text</p>
3 </div>
```

Привяжем событие к диву, но кликнем по абзацу. Очевидно, что клик по абзацу представляет собой одновременно и клик по диву, так как абзац содержится в нашем диве.

В описанном случае получится, что свойство target будет содержать конечный тег, в котором случилось событие - то есть абзац, а не див. Убедимся в этом:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function(event) {
4   console.log(event.target); // выведет наш абзац
5   console.log(this);        // выведет наш див
6 });
```

№ 1

Пусть у вас есть список `ul` с тегами `li`:

```
1  ul {  
2    padding: 30px;  
3    border: 1px solid red;  
4  }  
5  li {  
6    list-style-type: none;  
7    margin-bottom: 20px;  
8    border: 1px dashed black;  
9  }
```

```
1  <ul id="elem">  
2    <li>text</li>  
3    <li>text</li>  
4    <li>text</li>  
5    <li>text</li>  
6    <li>text</li>  
7  </ul>
```

Результат выполнения кода:

text

text

text

text

Привяжите к тегу `ul` обработчик клика. В этом обработчике проверяйте с помощью свойства `tagName`, по какому тегу был клик. Если клик был по тегу `li` - добавьте в конец текста этого тега восклицательный знак. А если клик был по тегу `ul` - выведите информацию об этом в консоль.

Получение нажатых клавиш в JavaScript

С помощью объекта `Event` можно получать нажатые клавиши. Давайте посмотрим, как это делается. Пусть у нас есть инпут:

```
1 <input>
```

Получим ссылку на него в переменную:

```
1 let elem = document.querySelector('input');
```

Навесим на наш инпут событие, срабатывающее по каждому нажатию клавиши:

```
1 elem.addEventListener('keypress', function(event) {  
2  
3 });
```

В свойстве `key` объекта с событием будет лежать нажатый символ:

```
1 elem.addEventListener('keypress', function(event) {  
2   console.log(event.key);  
3 });
```

А в свойстве `code` будет лежать код нажатой клавиши:

```
1 elem.addEventListener('keypress', function(event) {  
2   console.log(event.code);  
3 });
```

№ 1

⊗ jsPmOEPKG

Сделайте инпут, который будет по вводу выводить значения введенных клавиш и их коды.

№ 2

⊗ jsPmOEPKG

Понажимайте в инпуте различные клавиши и посмотрите, какие у них значения и коды.

№ 3

⊗ jsPmOEPKG

Определите, какой код будет у клавиши `Enter`.

№ 4

⊗ jsPmOEPKG

Определите, какой код будет у клавиши `BackSpace`.

№ 5

⊗ jsPmOEPKG

Дан абзац и инпут. В него вводится текст и нажимается клавиша `Enter`. Сделайте так, чтобы в этот момент введенный текст попадал в абзац под инпутом, а содержимое инпута очищалось.

Отслеживание клавиш-модификаторов в JavaScript

С помощью объекта `Event` можно узнать, были ли нажаты в момент события клавиши `Ctrl`, `Alt` и `Shift`. Это делается с помощью свойств `ctrlKey`, `altKey` и `shiftKey` - они имеют значение `true` или `false` в зависимости от того, была ли нажата эта клавиша в момент события или нет.

Давайте посмотрим на примере. Пусть у нас дана следующая кнопка:

```
1 <button id="elem">text</button>
```

По клику на кнопку будем выводить сообщение о том, была ли нажата одна из клавиш `Ctrl`, `Alt` и `Shift`:

```
1 let elem = document.querySelector('#elem');
2
3 elem.addEventListener('click', function(event) {
4   if (event.ctrlKey) {
5     console.log('нажат Ctrl');
6   }
7
8   if (event.altKey) {
9     console.log('нажат Alt');
10  }
11
12  if (event.shiftKey) {
13    console.log('нажат Shift');
14  }
15 });
```



№1

jsPmOEKM

Дан элемент. Сделайте так, чтобы по клику на него он красился в красный цвет, но только если в момент клика нажата клавиша `Alt`.

№2

jsPmOEKM

Пусть у вас есть список `ul` с тегами `li`:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Сделайте так, чтобы по клику на любую `li`, в конец ее текста добавлялось число 1, если нажата клавиша `Ctrl`, и число 2, если нажата клавиша `Shift`.

Отмена действия по умолчанию в JavaScript

Иногда с помощью JavaScript требуется отменить действие тега по умолчанию. К примеру, по нажатию на ссылку отменить переход по этой ссылке. Как правило такое требуется в том случае, если эту ссылку мы используем просто для запуска какого-либо кода. В этом случае то, что при клике на ссылку произойдет переход по ней, нам совсем не нужно.

Отмена действия по умолчанию производится с помощью объекта `Event`. Для этого у него есть специальный метод `preventDefault()`, который следует вызвать в любом месте обработчика события.

Попробуем на практике. Пусть у нас есть вот такая ссылка:

```
1 | <a href="/" id="elem">ссылка</a>
```

Давайте сделаем так, чтобы по клику на ссылку не происходило перехода на другую страницу:

```
1 | let elem = document.querySelector('#elem');
2 |
3 | elem.addEventListener('click', function(event) {
4 |   event.preventDefault();
5 |   console.log('Вы не можете перейти по этой ссылке!');
6 | });
```

№1

©jsPmOECDА

Даны ссылки. Сделайте так, чтобы по клику на ссылку ей в конец записывался ее href, а перехода по ссылке не происходило.

№2

©jsPmOECDА

Даны два инпута, абзац и ссылка. Пусть в инпуты вводятся числа. Сделайте так, чтобы по клику на ссылку в абзац записалась сумма введенных чисел.

Всплытие событий в JavaScript

Представьте себе, что у вас есть несколько вложенных друг в друга блоков:

```
1 | <div id="elem1">
2 |   <div id="elem2">
3 |     <div id="elem3"></div>
4 |   </div>
5 | </div>
```

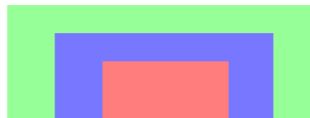
Получим ссылки на наши дивы в переменные:

```
1 | let elem1 = document.querySelector('#elem1');
2 | let elem2 = document.querySelector('#elem2');
3 | let elem3 = document.querySelector('#elem3');
```

А теперь навесим на них обработчики кликов:

```
1 | elem1.addEventListener('click', function() {
2 |   alert('зеленый');
3 | });
4 | elem2.addEventListener('click', function() {
5 |   alert('голубой');
6 | });
7 | elem3.addEventListener('click', function() {
8 |   alert('красный');
9 | });
```

Выведем наши блоки на экран, добавив им некоторый CSS:



Нажмите теперь на самый внутренний красный блок - и вы увидите, как событие сначала сработает в красном блоке, потом в голубом, потом в зеленом. И это логично, ведь кликая на внутренний блок, вы одновременно кликаете на все внешние.

То есть получается, что когда вы кликаете на самый внутренний блок, событие клика возникает сначала в нем, затем срабатывает в его родителе, в родителе его родителя и так далее, пока не дойдет до самого верха.

Такое поведение называется *всплытием* (bubbling) событий - по аналогии со всплытием пузырька воздуха со дна. Так же, как и пузырек, наше событие как будто выплывает наверх, каждый раз срабатывая на более высоких блоках.

№ 1

© jsPmOEEB

Самостоятельно напишите код, который будет демонстрировать всплытие событий. Проверьте на нем всплытие различных типов событий.

№ 2

© jsPmOEEB

Всплывать могут не все события. Экспериментально найдите хотя бы одно событие, которое не будет всплывать.

Получение целевого элемента при всплытии событий

Пусть у нас есть два элемента: div и абзац p, лежащий внутри этого дива:

```
1 <div>
2   <p></p>
3 </div>
```

Получим ссылку на наш див в переменную:

```
1 let div = document.querySelector('div');
```

Добавим нашим тегам некоторые стили:

```
1 div {
2   padding: 20px;
3   border: 1px solid red;
4 }
5
6 p {
7   width: 200px;
8   height: 200px;
9   border: 1px solid green;
10 }
```

Пусть теперь на див навешен обработчик клика:

```
1 div.addEventListener('click', function() {
2   console.log('click');
3 });
```

Из-за того, что наш див имеет padding, то, когда мы кликаем на див, мы можем попасть по абзацу, а можем попасть в место, где этого абзаца нет, то есть непосредственно по диву.

При этом в обработчике клика this всегда будет указывать на тот элемент, на который навешан обработчик. В нашем случае это наш див:

```

1 div.addEventListener('click', function() {
2   console.log(this); // див
3 });

```

Однако, мы можем получить именно тот тег, в котором случилось событие. Для этого мы можем посмотреть, что лежит в `event.target`:

```

1 div.addEventListener('click', function(event) {
2   console.log(event.target); // или див, или абзац
3 });

```

Можно различить эти два варианта с помощью условий:

```

1 let div = document.querySelector('div');
2
3 div.addEventListener('click', function(event) {
4   if (event.target.tagName === 'DIV') {
5     console.log('клик именно по диву');
6   }
7   if (event.target.tagName === 'P') {
8     console.log('клик именно по абзацу');
9   }
10 });

```

Можно вместо `tagName` использовать `matches`:

```

1 let div = document.querySelector('div');
2
3 div.addEventListener('click', function(event) {
4   if (event.target.matches('div')) {
5     console.log('клик именно по диву');
6   }
7   if (event.target.matches('p')) {
8     console.log('клик именно по абзацу');
9   }
10 });

```

№ 1

© jsPmOEBTE

Даны следующие элементы:

```

1 <div>
2   <ul>
3     <li>1</li>
4     <li>2</li>
5     <li>3</li>
6   </ul>
7 </div>

```

```

1 div, ul, li {
2   padding: 20px;
3 }
4
5 div {
6   border: 1px solid red;
7 }
8 ul {
9   border: 1px solid orange;
10 }
11 li {
12   border: 1px solid green;
13 }

```

Навешайте на див обработчик клика. В этом обработчике определите, в каком из тегов сработало событие.

№ 2

© jsPmOEBTE

Модифицируйте предыдущую задачу. Сделайте так, чтобы при клике на `li`, ей в конец добавлялся восклицательный знак, а при клике на `ul` в консоль выводилась информация об этом.

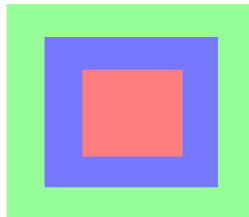
Прекращение всплытия событий в JavaScript

Всплытие события можно остановить на любой элемент, через который всплывает событие. Для этого в коде элемента следует вызвать метод `stopPropagation` объекта `Event`.

В следующем примере клик по красному блоку сработает на нем самом, затем на голубом блоке и все - голубой блок прекращает дальнейшее всплытие и зеленый блок уже никак не отреагирует:

```
1 elem1.addEventListener('click', function() {
2   console.log('зеленый');
3 });
4 elem2.addEventListener('click', function(event) {
5   console.log('голубой');
6   event.stopPropagation(); // остановим всплытие
7 });
8 elem3.addEventListener('click', function() {
9   console.log('красный');
10 });
```

Можете проверить:

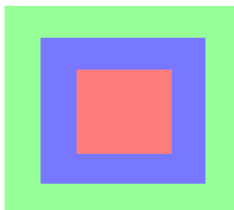


Несколько обработчиков на элементе в JavaScript

Если у элемента есть несколько обработчиков на одно событие, то даже при прекращении всплытия все они будут выполнены. То есть, `stopPropagation` препятствует продвижению события дальше, но на текущем элементе все обработчики отработают. Смотрите пример:

```
1 elem1.addEventListener('click', function() {
2   console.log('зеленый');
3 });
4 elem2.addEventListener('click', function(event) {
5   console.log('голубой - первый обработчик');
6   event.stopPropagation(); // остановим всплытие
7 });
8 elem2.addEventListener('click', function() {
9   console.log('голубой - второй обработчик'); // все равно работает
10 });
11 elem3.addEventListener('click', function() {
12   console.log('красный');
13 });
```

Можете проверить:

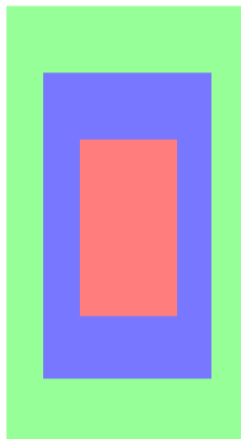


Немедленное прекращение всплытия в JavaScript

Для того, чтобы полностью остановить обработку, современные браузеры поддерживают метод `stopImmediatePropagation`. Он не только предотвращает всплытие, но и останавливает обработку событий на текущем элементе. Давайте его применим:

```
1 elem1.addEventListener('click', function() {  
2   console.log('зеленый');  
3 });  
4 elem2.addEventListener('click', function(event) {  
5   console.log('голубой - первый обработчик');  
6   event.stopImmediatePropagation();    // остановим всплытие  
7 });  
8 elem2.addEventListener('click', function() {  
9   console.log('голубой - второй обработчик'); // уже не сработает  
10 });  
11 elem3.addEventListener('click', function() {  
12   console.log('красный');  
13 });
```

Можете проверить:



Применение прекращения всплытия событий в JavaScript

Пусть внутри одного родителя у нас есть кнопка и некоторый блок:

```
1 <div id="parent">
2   <button>click me</button>
3   <div id="block">
4     text
5   </div>
6 </div>
```

Получим ссылки на наши элементы в переменные:

```
1 let parent = document.querySelector('#parent');
2 let button = document.querySelector('button');
3 let block = document.querySelector('#block');
```

Пусть наш блок изначально скрыт:

```
1 #block:not(.active) {
2   display: none;
3 }
```

Сделаем так, чтобы по клику на кнопку наш блок появился:

```
1 button.addEventListener('click', function() {
2   block.classList.add('active');
3 });
```

А теперь сделаем так, чтобы по клику на любое место родителя наш блок скрывался:

```
1 parent.addEventListener('click', function() {
2   block.classList.remove('active');
3 });
```

Нас, однако, ждет неожиданный сюрприз: так как кнопка находится внутри родителя, то *клик по кнопке одновременно означает и клик по родителю*. Это значит, что сначала наш блок покажется, а затем из-за всплытия события сработает обработчик в родителе и наш блок скроется.

Вот тут-то нам и пригодится возможность отменить всплытие: мы можем сделать так, чтобы при клике на кнопку отменить всплытие, родитель не реагировал на этот клик.

№1

⊗jsPmOEbUs

Самостоятельно реализуйте корректную работу описанной задачи.

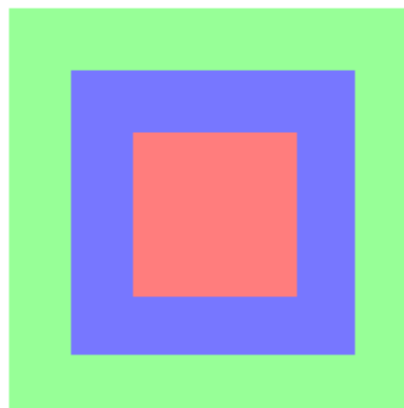
Погружение событий в JavaScript

Кроме всплытия событий есть еще и *погружение*. На самом деле событие сначала идет сверху вниз (стадия перехвата), доходит до нашего элемента (стадия цели) и только потом начинает всплывать (стадия всплытия).

Для того, чтобы повесить обработчик события с учетом стадии перехвата в `addEventListener` есть третий необязательный параметр. Если он равен `true` - событие сработает на стадии перехвата, а если `false` - на стадии всплытия (это по умолчанию). Давайте посмотрим на примере:

```
1 elem1.addEventListener('click', function() {  
2   console.log('зеленый - погружение');  
3 }, true);  
4 elem1.addEventListener('click', function() {  
5   console.log('зеленый - всплытие');  
6 }, false);  
7  
8 elem2.addEventListener('click', function() {  
9   console.log('голубой - погружение');  
10 }, true);  
11 elem2.addEventListener('click', function() {  
12   console.log('голубой - всплытие');  
13 }, false);  
14  
15 elem3.addEventListener('click', function() {  
16   console.log('красный - погружение');  
17 }, true);  
18 elem3.addEventListener('click', function() {  
19   console.log('красный - всплытие');  
20 }, false);
```

Можете проверить:



Навешивание обработчиков на новые элементы в JavaScript

Пусть у нас есть список `ul` и кнопка:

```
1 <ul>
2   <li>item</li>
3   <li>item</li>
4   <li>item</li>
5   <li>item</li>
6   <li>item</li>
7 </ul>
8
9 <button>add</button>
```

Получим наши элементы в соответствующие переменные:

```
1 let button = document.querySelector('button');
2 let list   = document.querySelector('ul');
3 let items  = list.querySelectorAll('li');
```

Сделаем так, чтобы по клику на любую `li` ей в конец добавлялся восклицательный знак:

```
1 for (let item of items) {
2   item.addEventListener('click', function() {
3     this.textContent = this.textContent + '!';
4   });
5 }
```

Давайте теперь сделаем так, чтобы по нажатию на кнопку в конец списка добавлялся новая `li`:

```
1 button.addEventListener('click', function() {
2   let item = document.createElement('li');
3   item.textContent = 'item';
4   list.appendChild(item);
5 });
```

Мы, однако, получим проблему: клик на новую добавленную li не будет приводить к добавлению восклицательного знака в конец. Дело в том, что мы добавляем обработчик клика только на те li, которые существовали изначально, но *не на новые*.

Исправим проблему, навесив на новую li обработчик клика:

```
1 button.addEventListener('click', function() {
2   let item = document.createElement('li');
3   item.textContent = 'item';
4
5   item.addEventListener('click', function() { // обработчик клика
6     this.textContent = this.textContent + '!';
7   });
8
9   list.appendChild(item);
10 });
```

Однако, теперь код функции-обработчика дублируется в двух местах - для изначально существующих li и для новых. Исправим это, вынеся его в отдельную функцию:

```
1 function handler() {
2   this.textContent = this.textContent + '!';
3 }
```

Используем нашу функцию, чтобы избежать дублирования кода:

```
1 for (let item of items) {
2   item.addEventListener('click', handler);
3 }
4
5 button.addEventListener('click', function() {
6   let item = document.createElement('li');
7   item.textContent = 'item';
8
9   item.addEventListener('click', handler);
10
11   list.appendChild(item);
12 });
```

Задача в общем-то решена и мы избежали дублирования кода функции-обработчика. Однако, навешивать обработчики событий нам все равно приходится в двух местах: и в цикле для существующих li, и в при клике на кнопку. В следующем уроке мы разберем способ избавиться от этого неудобства.

Делегирование событий в JavaScript

В предыдущем уроке была описана проблема, возникающая при добавлении новых элементов, и приведено ее решение. В данном уроке мы рассмотрим более удачный способ обойти проблему - *делегирование событий*. Давайте его разберем.

Как вы уже знаете, кликая на li, мы одновременно кликаем и на ul. Это возможно благодаря всплытию событий. Мы можем использовать это для решения нашей задачи: навесим событие не на каждую li, а на их родителя ul:

```
1 list.addEventListener('click', function() {  
2  
3 });
```

Теперь в обработчике события this будет указывать на элемент, к которому привязан обработчик, а event.target - на элемент, в котором случилось событие:

```
1 list.addEventListener('click', function(event) {  
2   console.log(this); // наш список  
3   console.log(event.target); // пункт списка  
4 });
```

Сделаем так, чтобы li, на которой случился клик, в конец добавлялся восклицательный знак:

```
1 list.addEventListener('click', function(event) {  
2   event.target.textContent = event.target.textContent + '!';  
3 });
```

№ 1

©jsPmOEED

Повторите приведенное решение. Убедитесь, что новые li также будут реагировать на клик.

Универсальное делегирование событий в JavaScript

Приведенный в предыдущем уроке код рабочий, однако, не без недостатков. Давайте разберем эти недостатки и напишем более универсальное решение.

Недостаток нашего кода проявится в том случае, когда внутри li будут какие-то вложенные теги. В нашем случае пусть это будут теги i:

```
1 <ul>  
2   <li>item <i>italic</i> item</li>  
3   <li>item <i>italic</i> item</li>  
4   <li>item <i>italic</i> item</li>  
5   <li>item <i>italic</i> item</li>  
6   <li>item <i>italic</i> item</li>  
7 </ul>
```

В этом случае нажатие на теги i приведет к добавлению восклицательного знака в *конец тега i*, а не тега li, как мы хотели бы - ведь в event.target попадает именно тот тег, в котором случилось событие.

Решить проблему можно с помощью метода closest:

```
1 list.addEventListener('click', function(event) {  
2   let li = event.target.closest('li');  
3  
4   if (li) {  
5     li.innerHTML = li.innerHTML + '!';  
6   }  
7 });
```

№ 1

©jsPmOEUED

Повторите приведенное решение.